Parte 1 - Aquisição e resiliência na borda

CardioIA Conectada - Fase 3 / Capítulo 1
Guilherme Yamada Dantas - RM rm568506
Data: 30/10/2026. Protótipo acadêmico com dados sintéticos.

Arquitetura e aquisição

O firmware foi preparado para ESP32 DevKit no Wokwi. Dois sensores distintos atendem à aquisição: DHT22 para temperatura e umidade, e MPU6050 para aceleração. O DHT22 conta como um único sensor, conforme o enunciado. Um potenciômetro adicional no GPIO34 controla uma frequência cardíaca **sintética** entre 40 e 180 bpm; ele não mede pulso. Temperatura e aceleração são estímulos do simulador e não medições de pacientes. O DHT22 tampouco constitui um termômetro clínico validado: a demonstração usa o valor configurado no simulador como variável de cenário.

O DHT22 usa GPIO15; o MPU6050 usa I²C SDA21/SCL22, alimentados por 3,3 V com terra comum. A cada cinco segundos o coletor registra sequência, tempo desde o boot, temperatura em °C, umidade em %, BPM sintético e magnitude da aceleração em m/s². Valores não finitos ou fora dos intervalos do DHT22 são representados como `null`, acompanhados por `sensor_error=true`, em vez de produzir JSON inválido ou um valor normal fictício. A aceleração inclui gravidade, portanto cerca de 9,81 m/s² parado não significa movimento. Não há diagnóstico de queda ou inatividade.

Estratégia limitada de resiliência

Uma fila circular na RAM reserva exatamente 2.000 registros. À taxa de uma amostra por cinco segundos, isso representa aproximadamente 10.000 segundos (2 h 46 min 40 s) de capacidade de retenção. O intervalo entre a primeira e a última das 2.000 leituras é 9.995 segundos. Quando chega a 2.001ª leitura não enviada, a mais antiga é descartada, preservando as 2.000 mais recentes. O contador `dropped` torna explícito o número de descartes da fila. Não mede perda de transporte: uma amostra em trânsito pode ser descartada enquanto sua publicação termina, e QoS 0 pode perder mensagens sem incrementar esse contador. A escolha privilegia a situação recente do cenário de acompanhamento, mas não é adequada a um histórico clínico que exija completude.

O comando `0` no Monitor Serial define a variável booleana `simulatedOnline=false`: a coleta continua e o cliente MQTT encerra a conexão. `1` habilita novamente a transmissão; `s` mostra tamanho e perdas da fila. Uma operação de rede já iniciada pode terminar antes de reconhecer `0`. Reconexão TCP/DNS/MQTT acontece em uma tarefa FreeRTOS separada; o coletor não espera pelo broker. A fila é protegida por mutex, liberado antes da operação de rede, e a remoção verifica a sequência para não apagar uma leitura diferente caso ocorra overflow durante um envio.

Ao reconectar, a tarefa publica os registros em ordem, até dez por segundo, espelha cada publicação no Serial e remove somente o registro cujo envio retornou sucesso. O protocolo usa MQTT QoS 0: sucesso de `publish()` significa escrita aceita pela biblioteca e não confirmação do broker nem processamento no dashboard. Portanto, ainda existe uma janela de perda após remoção; não se afirma entrega garantida. A identificação composta `device_id/session_id/seq` permite ao consumidor distinguir reinicializações e duplicatas. `uptime_ms` não é hora civil e não deve ser apresentado como horário clínico. O campo `age_ms`, calculado imediatamente antes da publicação pela subtração unsigned `millis() - uptime_ms`, indica a idade da amostra no dispositivo e permite distinguir backlog de leituras recentes. Não inclui o atraso posterior de rede; o dashboard deve considerar essa idade para não apresentar leituras antigas como atuais.

Limitações e verificação

SPIFFS é opcional no enunciado e não foi implementado. A fila é volátil: reiniciar o ESP32 ou encerrar a simulação perde as amostras pendentes. O Serial constitui evidência observável, mas não é um banco persistente automático. Uma evolução física exigiria armazenamento persistente com política de desgaste e confirmações de entrega pela aplicação. O broker público em porta 1883 não usa autenticação nem TLS; serve exclusivamente a dados sintéticos desta demonstração, sem identificadores de pacientes. O `RM` é identificação acadêmica do projeto.

A implementação inclui teste C++ host do mesmo buffer utilizado no firmware: fila vazia, FIFO, retorno circular, overflow de 10.000 inserções mantendo as últimas 2.000, contagem de 8.000 perdas e proteção contra remoção indevida de mensagem em trânsito. A compilação ESP32 foi concluída com sucesso: RAM estática de 93.380 bytes (28,5%) e flash de 785.065 bytes (59,9%). A RAM dinâmica não foi medida. Essa compilação e o teste do buffer são verificações técnicas distintas de uma execução Wokwi. O estado e logs reais ficam em `firmware/evidence/`. A execução Wokwi foi observada: coleta válida, quatro amostras acumuladas offline e reenvio após comando 1, com recepção MQTT no Node-RED. O projeto foi salvo em https://wokwi.com/projects/479273729983469569. Logs e capturas estão em evidence/wokwi-final-*. Não houve execução em hardware físico.

Referências utilizadas

- FIAP. Enunciado local: Fase 3, Capítulo 1, CardioIA Conectada. Parte 1.
- FIAP. Cap08, *Cloud Computing como Pilar da IoT Moderna*, seções 2.2.1 e 2.2.2: ESP32, MPU6050 e comunicação MQTT.
- FIAP. Cap09, *Inteligência na Borda: Fog e Edge Computing Transformando a IoT*, seções 1.1 e 3: processamento próximo à origem e limites de arquiteturas distribuídas.
- Wokwi. MPU6050: https://docs.wokwi.com/parts/wokwi-mpu6050
- PubSubClient. Código de publicação e timeout: https://github.com/knolleary/pubsubclient/blob/v2.8/src/PubSubClient.cpp